

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 04

Студент: Стелина Петрити

Группа: НПИбд-02-21

Код:

```
#include <SFML/Graphics.hpp>
#include <cmath>
const double M_PI = 3.14159265358979323846;

// Класс Vertex для представления трехмерной точки в мировых и видовых координатах
class Vertex {
public:
    sf::Vector3f worldCoordinates, viewCoordinates;
    double screenX, screenY;

    Vertex(double x, double y, double z) : worldCoordinates(x, y, z) {}

    // Установка видовых координат на основе углов поворота (F и T) и расстояния (ro)
    void setViewCoordinates(double rotationF, double rotationT, double distanceRo) {
        // Упрощенная матрица трансформации для поворота и трансляции
        double sinT = sin(rotationT), cosT = cos(rotationT);
        double sinF = sin(rotationF), cosF = cos(rotationF);

        // Матрица трансформации
        double transformationMatrix[4][4] = {
            {-sinT, cosT, 0.0, 0.0},
            {-cosF * cosT, -cosF * sinT, sinF, 0.0},
            {-sinF * cosT, -sinF * sinT, -cosF, distanceRo},
            {0.0, 0.0, 0.0, 1.0}
        };

        // Применение трансформации для преобразования мировых координат в видовые
        double arrWorld[4] = { worldCoordinates.x, worldCoordinates.y,
worldCoordinates.z, 1.0 };
        double arrView[4] = { 0.0 };

        for (int i = 0; i < 4; ++i) {
            for (int j = 0; j < 4; ++j) {
                arrView[i] += transformationMatrix[i][j] * arrWorld[j];
            }
        }

        viewCoordinates.x = arrView[0];
        viewCoordinates.y = arrView[1];
        viewCoordinates.z = arrView[2];

        // Проецирование на двумерные координаты (упрощенная перспективная проекция)
        screenX = distanceRo / (2 * viewCoordinates.z) * viewCoordinates.x;
        screenY = distanceRo / (2 * viewCoordinates.z) * viewCoordinates.y;

        // Применение преобразования вьюпорта (упрощенное)
        screenX = 500 + screenX;
        screenY = 400 - screenY;
    }
};

// Класс Triangle для представления трехмерного треугольника
```

```

class Triangle {
public:
    Vertex vertex1, vertex2, vertex3;

    // Добавлен default-конструктор, инициализирующий вершины треугольника
    Triangle() : vertex1(0, 0, 0), vertex2(0, 0, 0), vertex3(0, 0, 0) {}

    Triangle(const Vertex& v1, const Vertex& v2, const Vertex& v3) : vertex1(v1),
vertex2(v2), vertex3(v3) {}

    // Нарисовать треугольник только в том случае, если он обращен к наблюдателю
    (отсечение задних граней)
    void draw(sf::RenderWindow& window) {
        if (calculateH() > 0.0) {
            // Определение линий для рисования
            sf::Vertex line1[] = {
                sf::Vertex(sf::Vector2f(vertex1.screenX, vertex1.screenY)),
                sf::Vertex(sf::Vector2f(vertex2.screenX, vertex2.screenY))
            };

            sf::Vertex line2[] = {
                sf::Vertex(sf::Vector2f(vertex3.screenX, vertex3.screenY)),
                sf::Vertex(sf::Vector2f(vertex2.screenX, vertex2.screenY))
            };

            // Рисование линий
            window.draw(line1, 2, sf::Lines);
            window.draw(line2, 2, sf::Lines);
        }
    }

private:
    // Вычислить значение H для отсечения задних граней
    double calculateH() const {
        double x1 = vertex1.viewCoordinates.x, y1 = vertex1.viewCoordinates.y, z1 =
vertex1.viewCoordinates.z;
        double x2 = vertex2.viewCoordinates.x, y2 = vertex2.viewCoordinates.y, z2 =
vertex2.viewCoordinates.z;
        double x3 = vertex3.viewCoordinates.x, y3 = vertex3.viewCoordinates.y, z3 =
vertex3.viewCoordinates.z;

        return x1 * (y2 * z3 - y3 * z2) - x2 * (y1 * z3 - y3 * z1) + x3 * (y1 * z2 -
y2 * z1);
    }
};

int main() {
    sf::RenderWindow window(sf::VideoMode(1000, 800), "Куб");

    // Создание вершин куба
    Vertex cubeVertices[] = {
        Vertex(-100.0, -100.0, -100.0),
        Vertex(100.0, -100.0, -100.0),
        Vertex(100.0, 100.0, -100.0),
        Vertex(-100.0, 100.0, -100.0),
        Vertex(-100.0, -100.0, 100.0),
        Vertex(100.0, -100.0, 100.0),
        Vertex(100.0, 100.0, 100.0),
        Vertex(-100.0, 100.0, 100.0)
    };
}

```

```

    Vertex(-100.0, 100.0, 100.0)
};

// Создание треугольников куба
Triangle cubeFaces[12];

// Индексы вершин для каждого треугольника
int indices[][3] = {
    {4, 0, 1}, {1, 5, 4},
    {5, 1, 2}, {2, 6, 5},
    {6, 2, 3}, {3, 7, 6},
    {7, 3, 0}, {0, 4, 7},
    {4, 5, 6}, {6, 7, 4},
    {0, 3, 2}, {2, 1, 0}
};

// Заполнение массива треугольников
for (int i = 0; i < 12; ++i) {
    cubeFaces[i] = Triangle(cubeVertices[indices[i][0]],
        cubeVertices[indices[i][1]], cubeVertices[indices[i][2]]);
}

double rotationF = 0, rotationT = 0, distanceRo = 500;

while (window.isOpen()) {
    sf::Event event;
    while (window.pollEvent(event)) {
        if (event.type == sf::Event::Closed)
            window.close();
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right))
            rotationT += 10;
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left))
            rotationT -= 10;
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up))
            rotationF += 10;
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down))
            rotationF -= 10;
    }

    window.clear();

    // Установка видовых координат для каждой вершины на основе ввода
    пользователя
    for (auto& face : cubeFaces) {
        face.vertex1.setViewCoordinates(rotationF * M_PI / 90.0, rotationT *
M_PI / 90.0, distanceRo);
        face.vertex2.setViewCoordinates(rotationF * M_PI / 90.0, rotationT *
M_PI / 90.0, distanceRo);
        face.vertex3.setViewCoordinates(rotationF * M_PI / 90.0, rotationT *
M_PI / 90.0, distanceRo);

        // Нарисовать треугольник
        face.draw(window);
    }

    window.display();
}

```

```
    return 0;  
}
```

Вывод:

